2025S CPE 690-A - Introduction to VLSI Design
Homework 6
Guillaume Wantiez

Part 1: 8 inputs NAND gate schematic

Creation

The first part of the project will be to draw the electrical schematic for the 8 input NAND gate, since we are given specific size for the transistor (NMOS = 2 and PMOS = 8), we would not be able to design a multilevel logic, hence we will stay focused on a one level schematic.

We are trying to draw a transistor level schematic that will realize the function :

F = ¬(ABCDEFGH)

The PDN will realize the function:
F_{PDN} = ABCDEFGH

The PUN will realize the function:
F_{PUN} = A+B+C+D+E+F+G+H

So using cadence we will need to draw the following schematic:

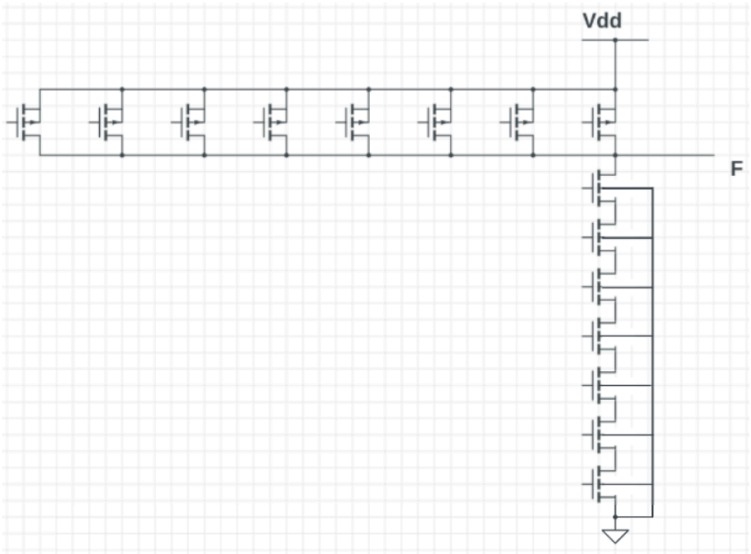

Figure 1.1 - CMOS transistor level schematic of F = not(ABCDEFGH)

First step is to create a new library for our project that I will call “8-input-NAND”, then create a new cellview called “Schematic”:

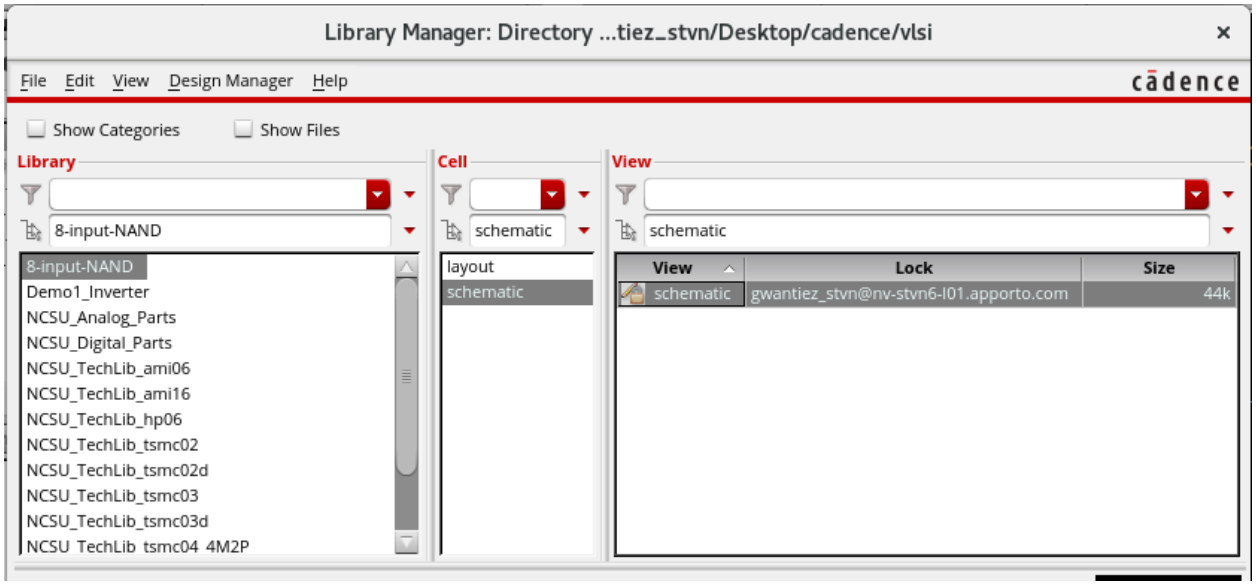

Figure 1.2 - Library Manager after the creation of the new library and schematic cellview

Once on the blank sheet, we need to import the following:

- 8 Nmos transistors (labeled nmos 4 in the analog component library). We will size them 8*1.5u = 12 u (since 1.5u = 1) for the PDN, using the property manager.

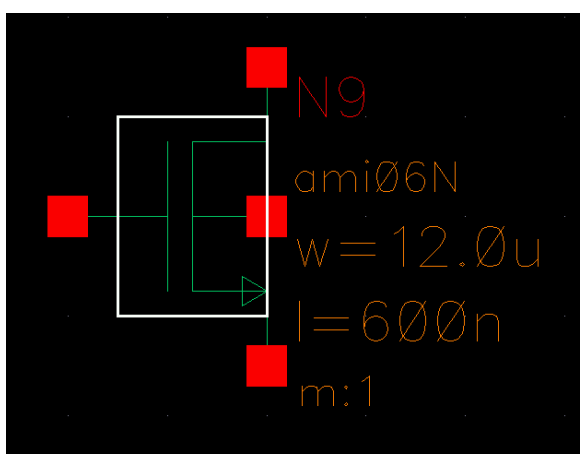

Figure 1.3 - Example of 8-unit width Nmos transistor used in the schematic

- 8 Pmos transistors (labeled nmos 4 in the analog component library). We will size them 2*1.5u = 3u, for the PUN, using the property manager.

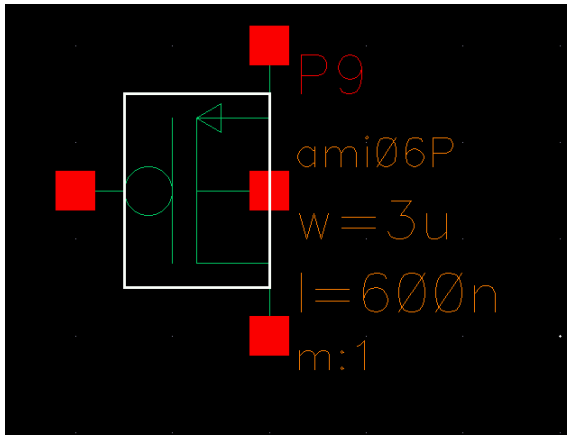

Figure 1.4 - Example of 2-unit width Pmos transistor used in the schematic

- One VDC source to simulate VDD.

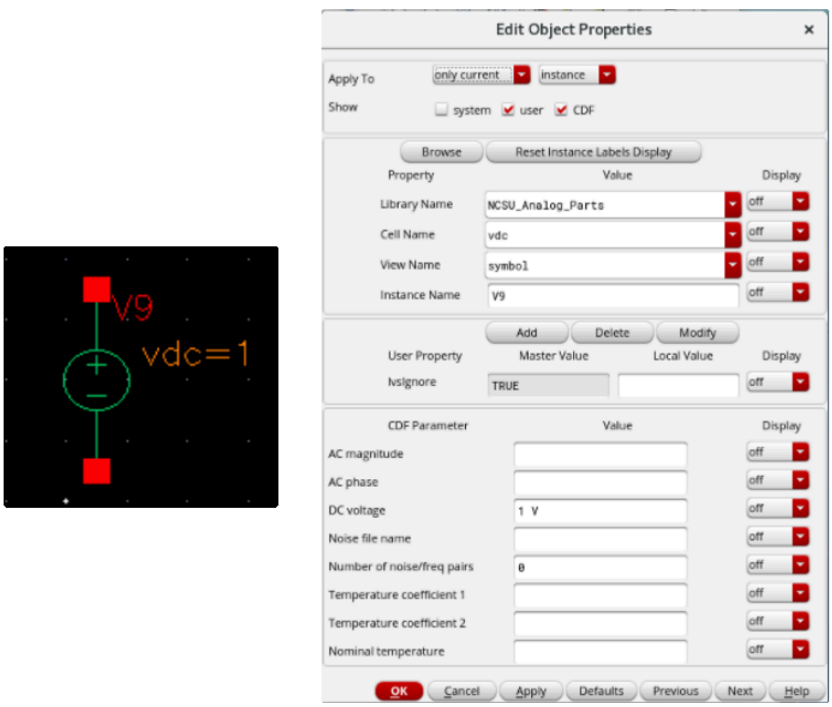

Figure 1.5 - VDC source used in the schematic and its properties

- We will need an output pin for F and to simulate each input we will use Vpulse power sources that we will edit between simulations.

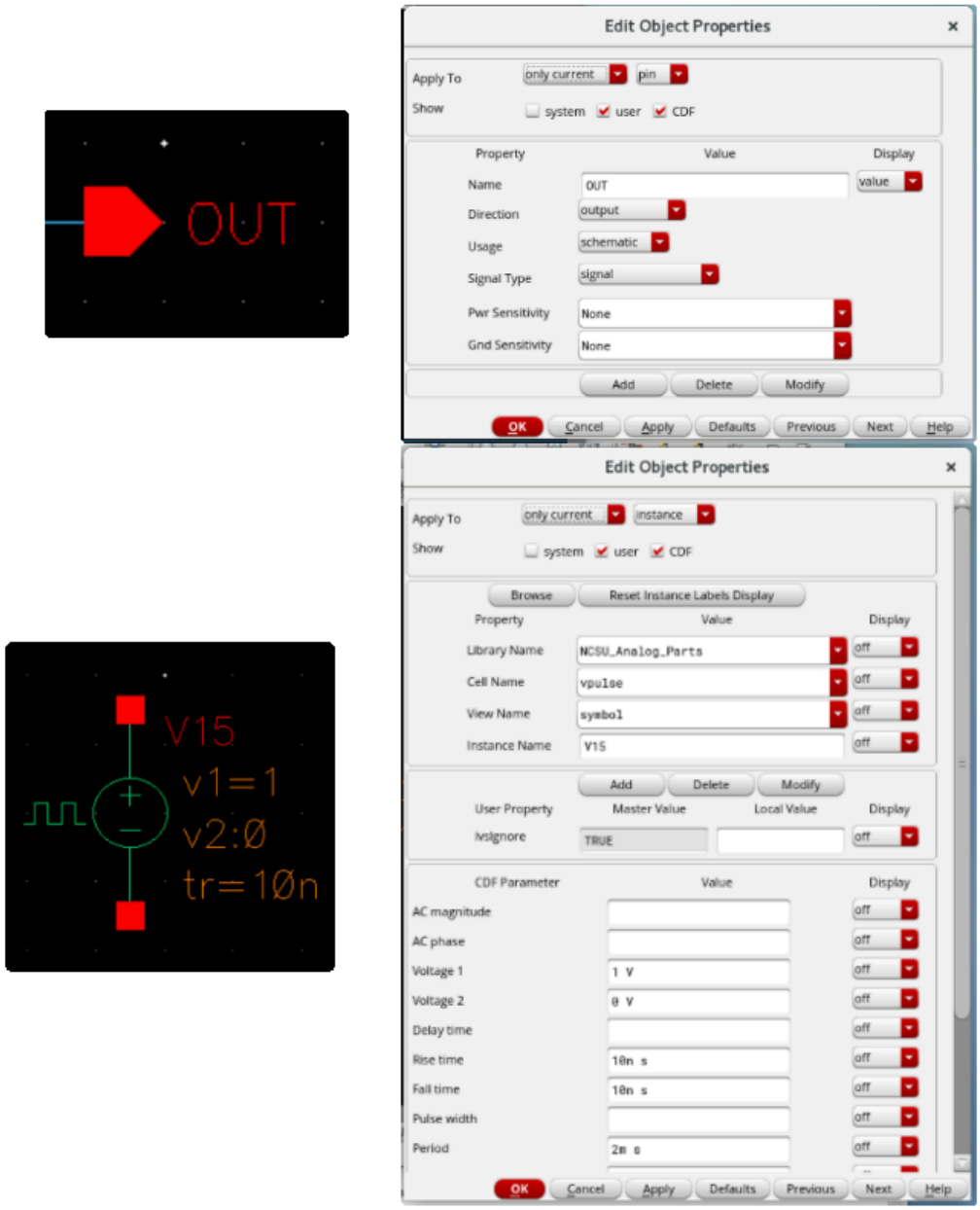

Figure 1.6 - OUT pin and Vpulse source used in the schematic and their properties

Next steps are to link all the components. We first place all the Pmos in parallel, connect them to the VDC and to the OUT pin, and connect the body pin of each transistor to their source pin (which is attached to VDC), giving us the following PUN:

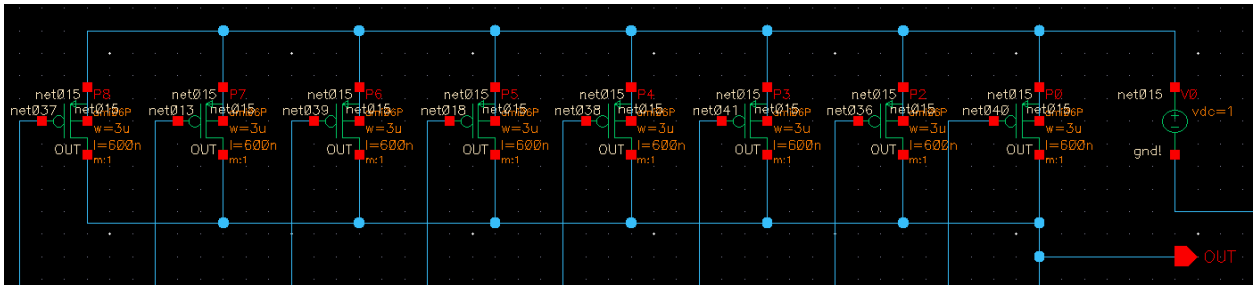

Figure 1.7 - PUN

Then we craft the PDN by placing all Nmos in series, connecting them together, and connecting the body pin of each transistor to the GND.

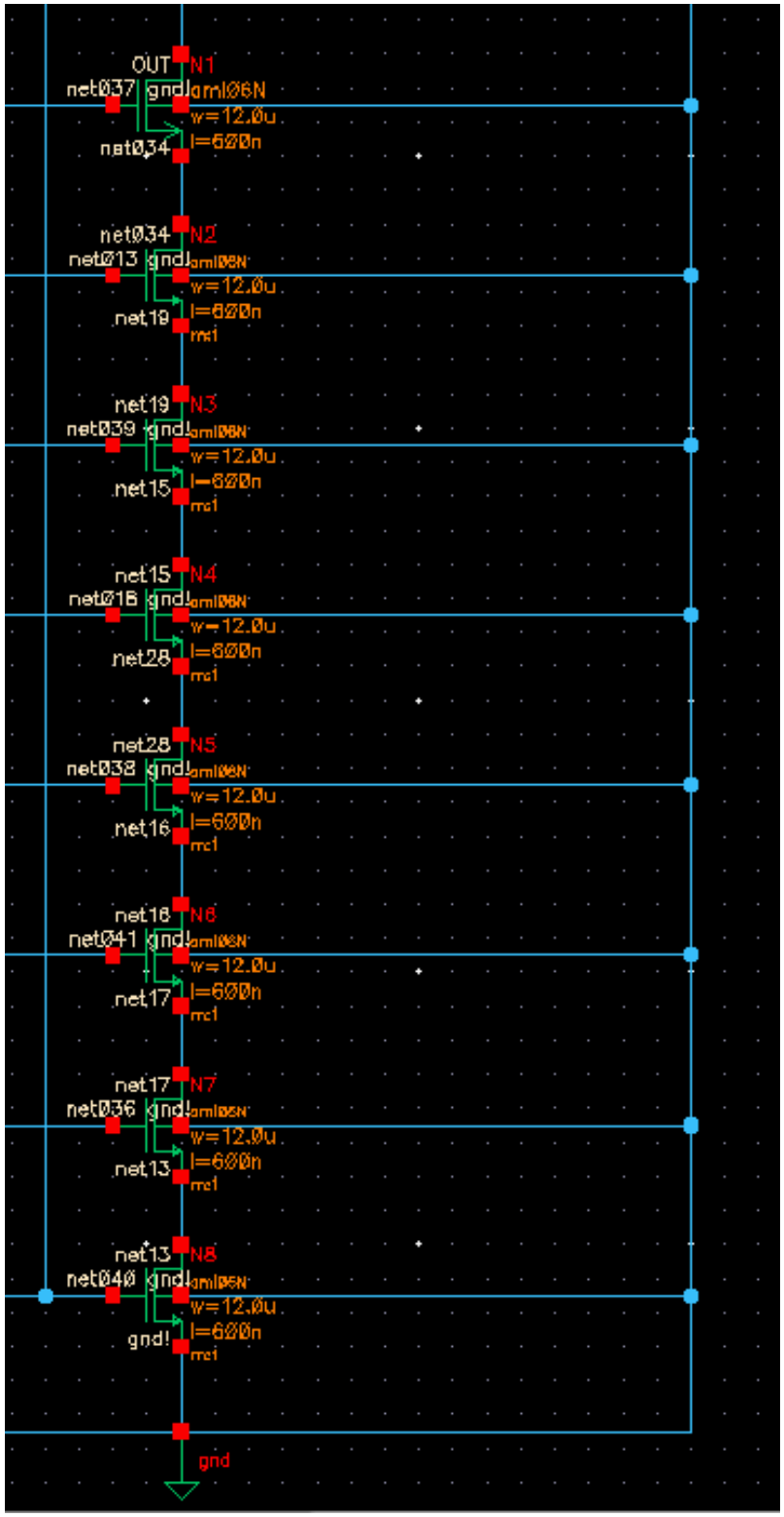

Figure 1.8 - PDN

Each input will be simulated using a pulse signal, connected to the source of a Pmos and Nmos and to the Gnd. The final schematic will be:

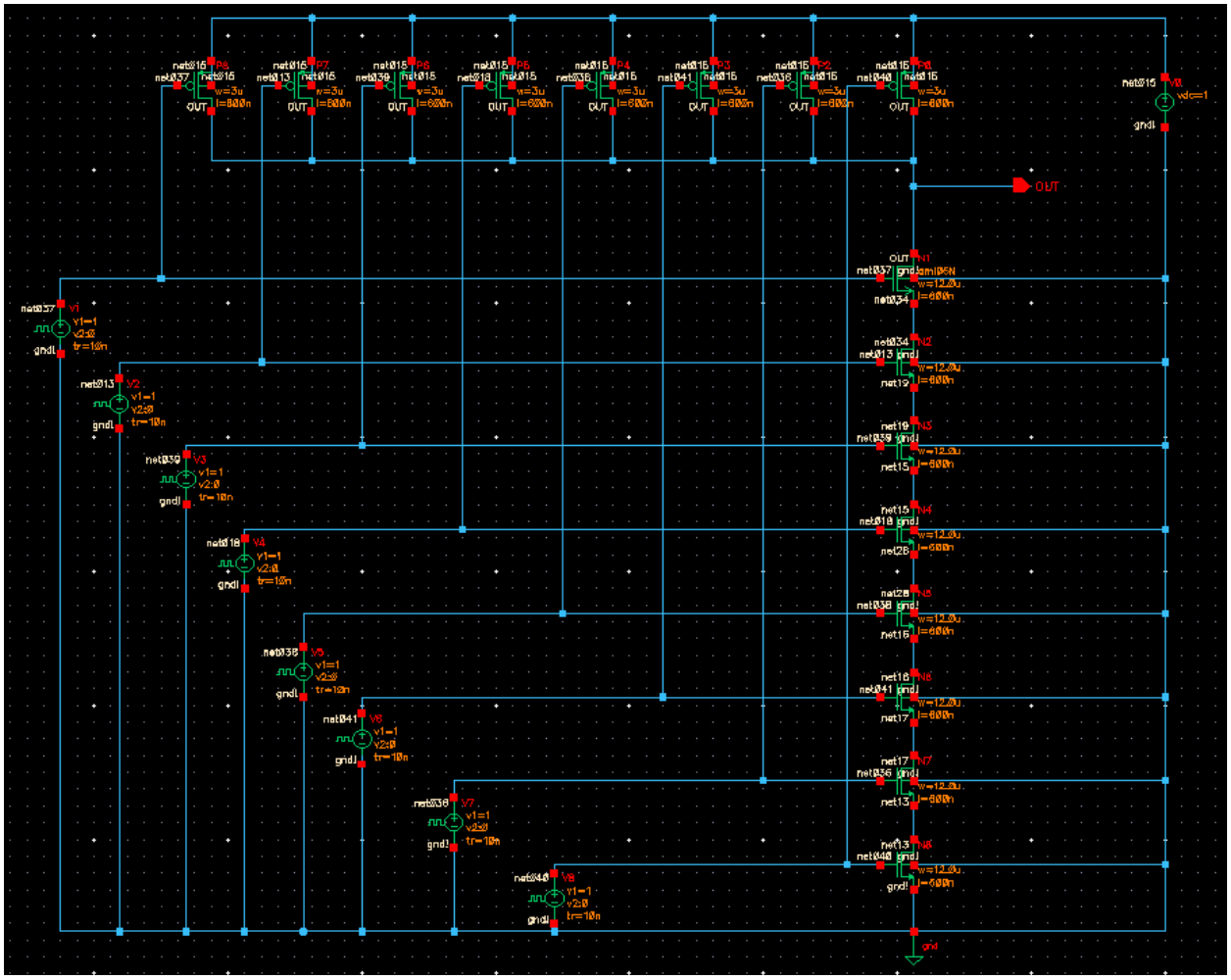

Figure 1.9 - 8 input NAND gate transistor level schematic made on Cadence virtuoso

Checking and Simulations

In this part we will first check that the schematic is correct and conduct an ADE L simulation with Specter to see if the circuit works as intended.

First let's hit Check and Save :

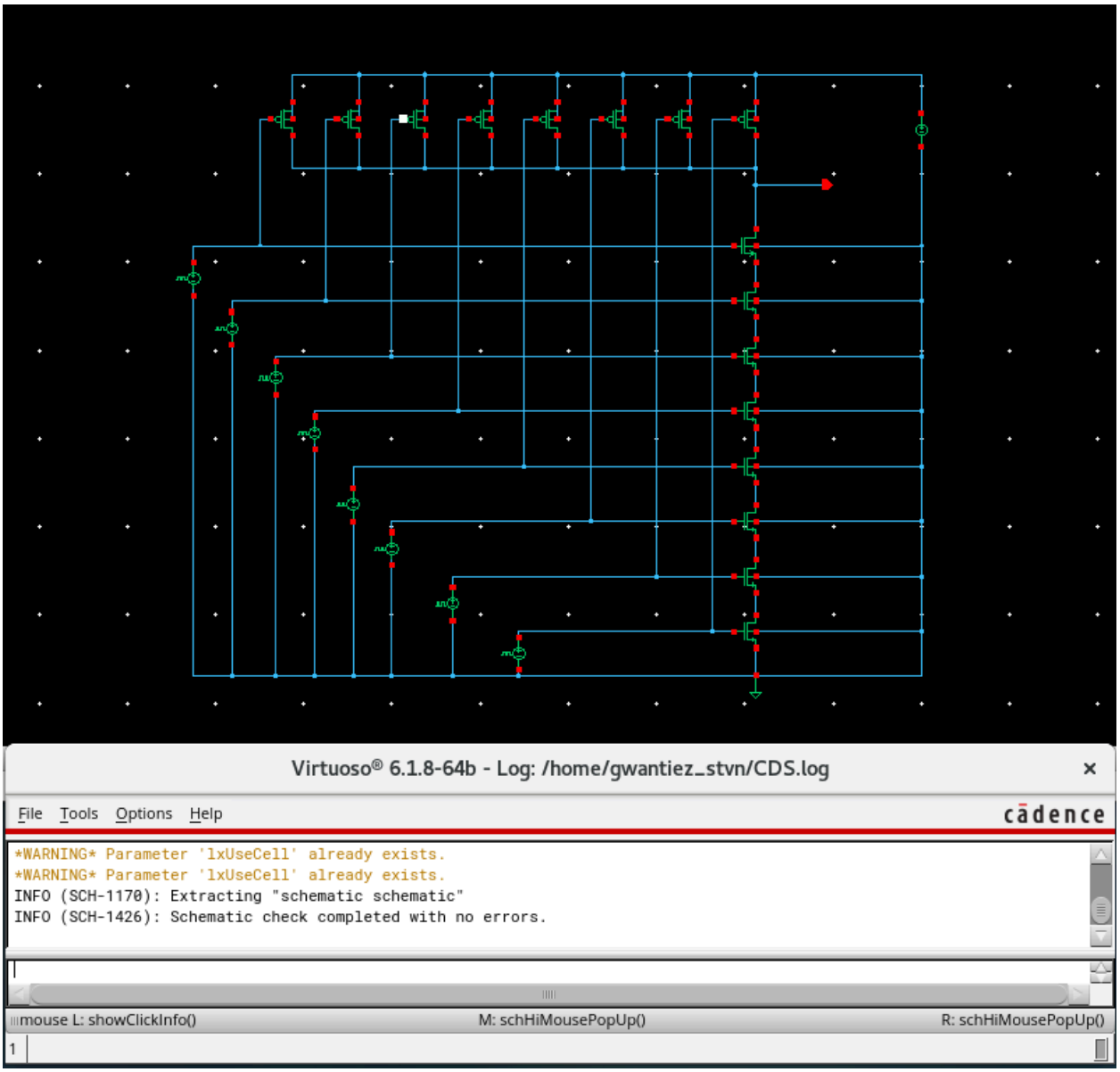

Figure 1.10 - CDS log window showing that the schematic is correct

We have the confirmation that the schematic is valid, now we want to know if it’s correct, let’s run several ADE L simulations to check it. To do it, let's follow the tutorials on how to set up the specter simulator (loading the proper components models, setting up the plot for the output, and inputs, setting specter as our simulator).

Here is the final ADE L state we will use:

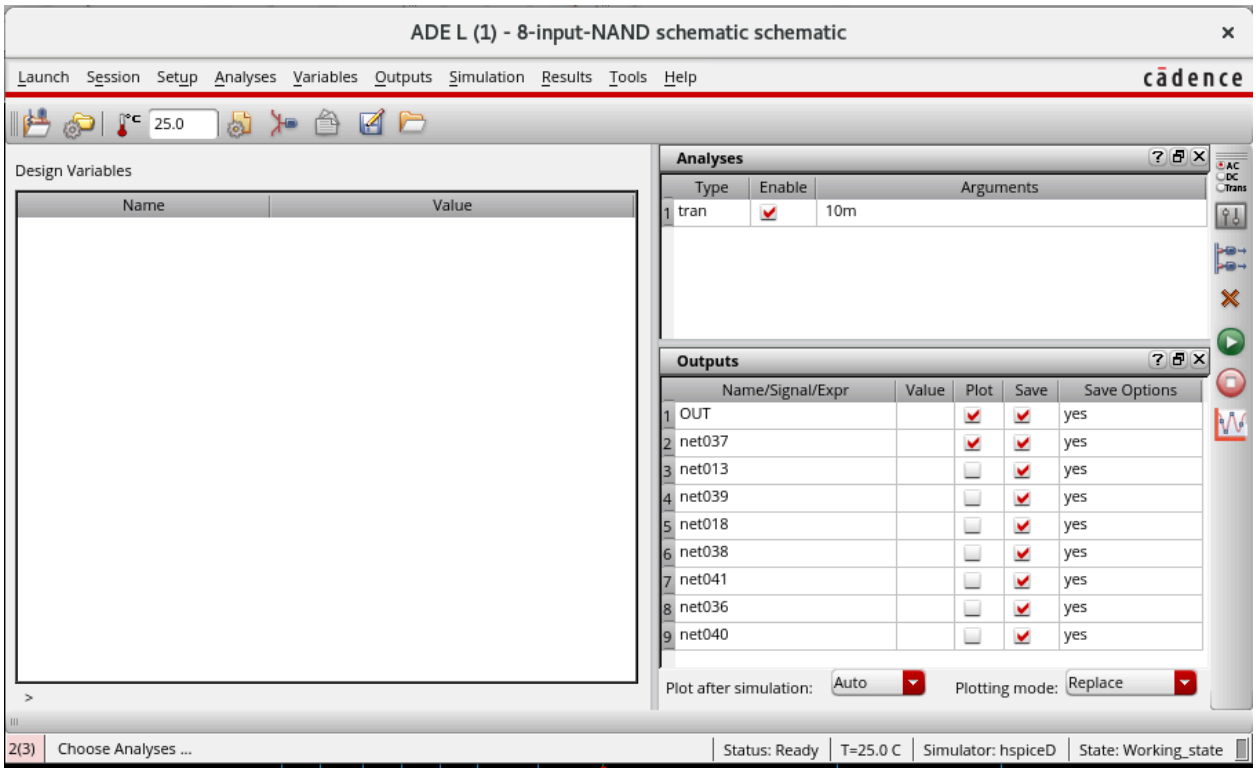

Figure 1.11 - ADE L final state

Here we can see that we will plot only one input and the output and that we will run a transient analysis that will run during 10 ms. Here's some results showing that the gate is working as intended:

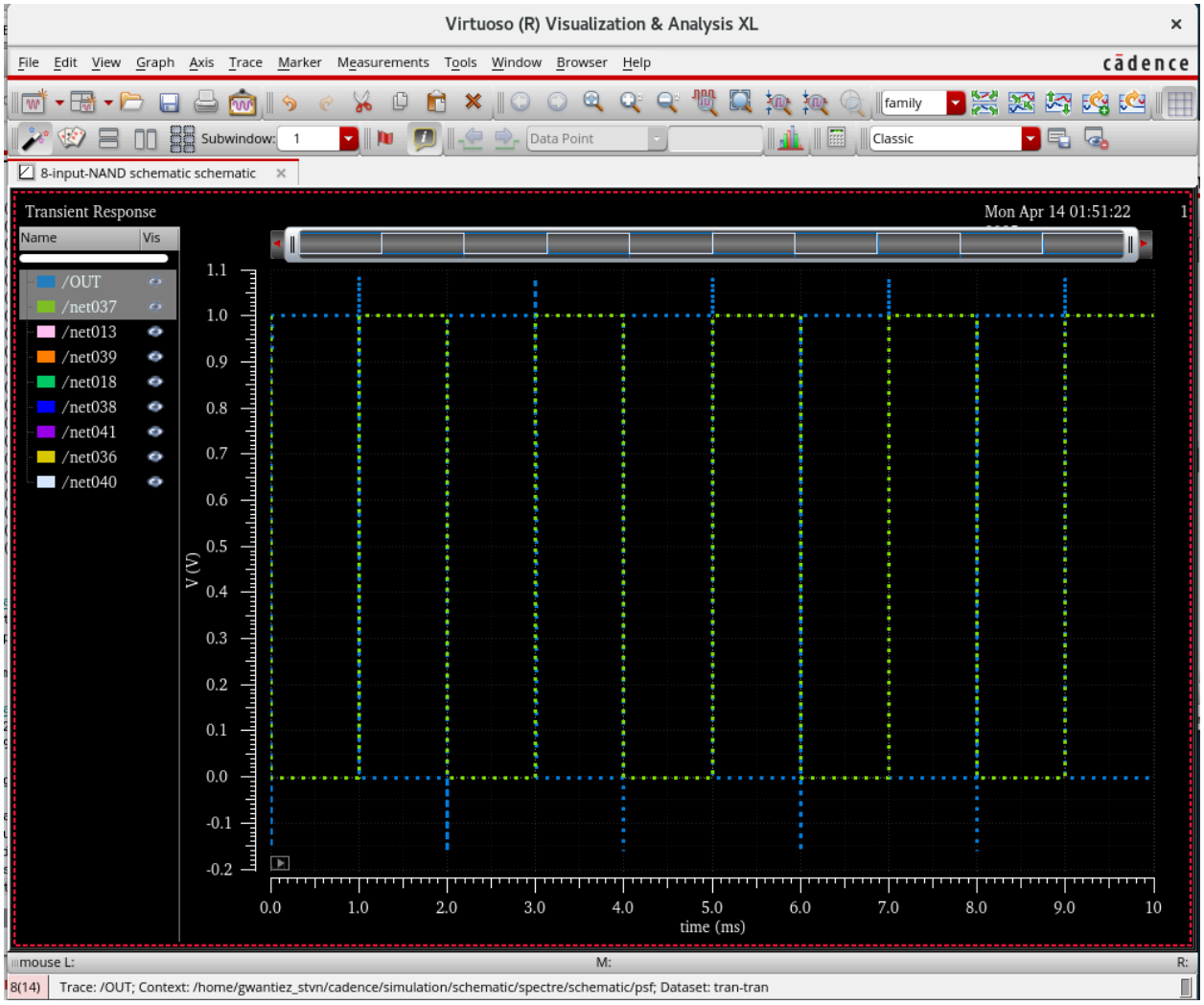

Figure 1.12 - Simulation showing OUT switching to 1 when all inputs are 0

On the above screenshot, we can see that each time the green signal (A) goes to 0, OUT (F) goes to 1. The other inputs are superposed with A meaning that the input signal is switching from all 0 to all 1.

The NAND truth table is the following:

A	B	Output
0	0	1
1	0	1
0	1	1
1	1	0

Figure 1.13 - 2 input NAND gate truth table

Until now the simulation seems to follow the logic of an 8 input NAND, let's try to simulate all the input states at once and see when the OUT signal is. To do so we will set a period for the first input, divide it by 2 and set this new period for the second input. We will do this until all 8 vpulse are set.

Let’s simulate the schematic behavior:

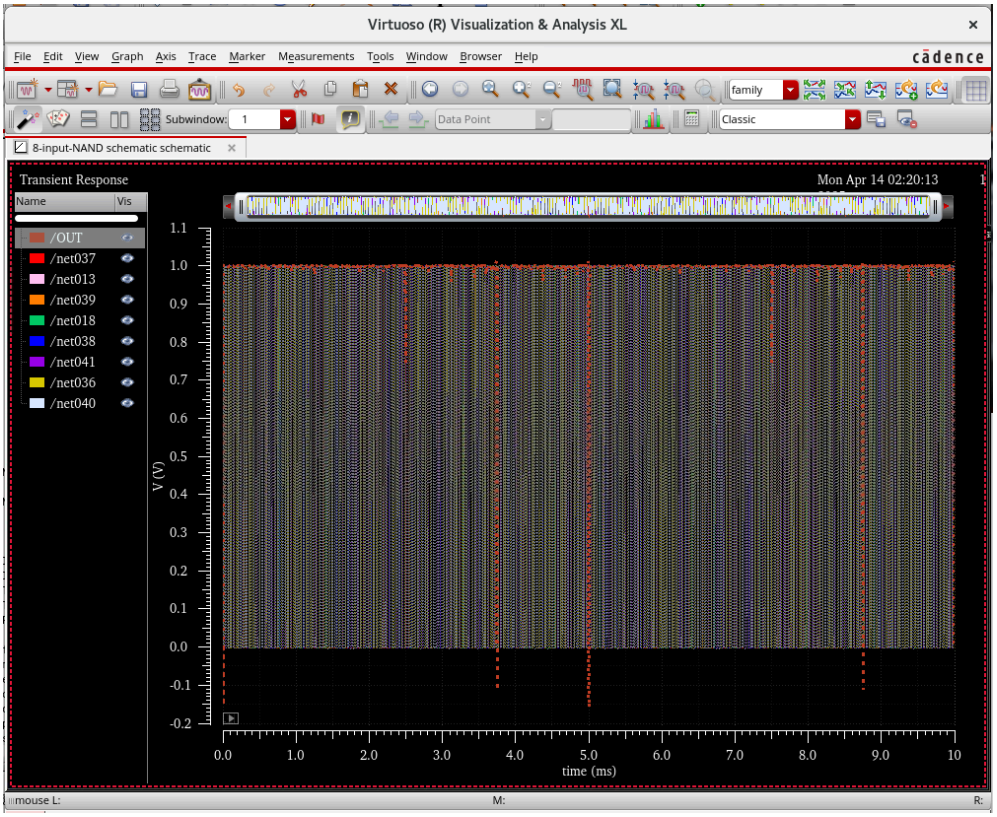

Figure 1.14 - First simulation with all possible inputs (Not clear)

As we can see it’s hard to understand a thing as it. We can refine the result by zooming in each time OUT is switching below 0, that would mean that OUT held a strong 0. To get a cleaner view of the output reaction we can split inputs and outputs on the analysis window.

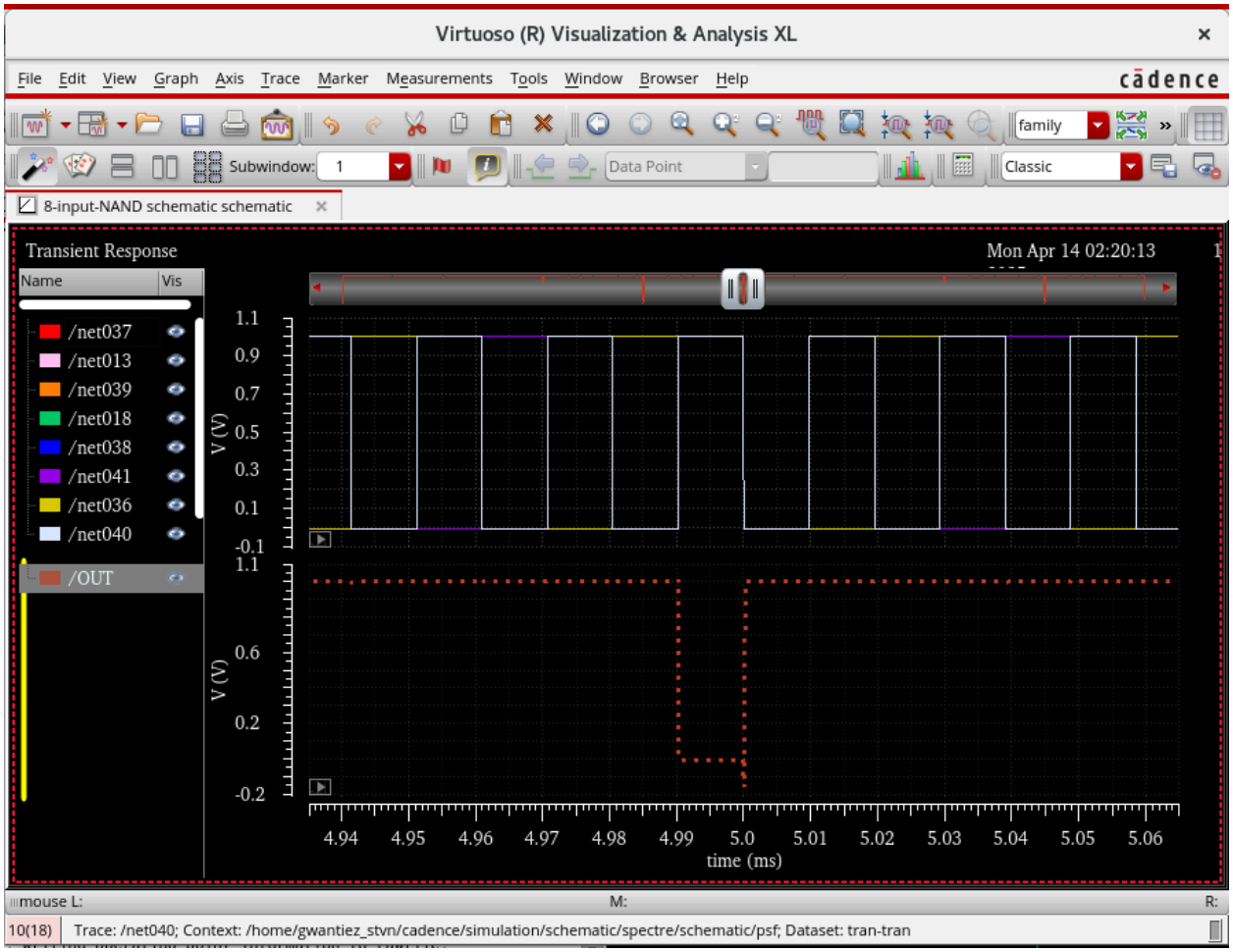

Figure 1.15 - Refined view of the simulation showing OUT reaction vs Inputs signals

As we can see OUT is falling when all inputs are synchronized to 1 otherwise OUT is 0. For all other OUT down spikes, the inputs are synchronized to 1 while OUT is 0, meaning that our schematic is fully functional.

<u>Note that this bunch of simulations are not taking any input capacitances loads or anything into account, it is just to achieve the proper logic before going into delay simulations that will be discussed in part 3.</u>

Part 2: 8 inputs NAND gate layout and LVS

Drawing the layout

Now that we have a fully functional schematic, the next step is to draw the layout of our logical gate. We will start with the inverter layout we made in the cadence tutorial (see below) and refine it.

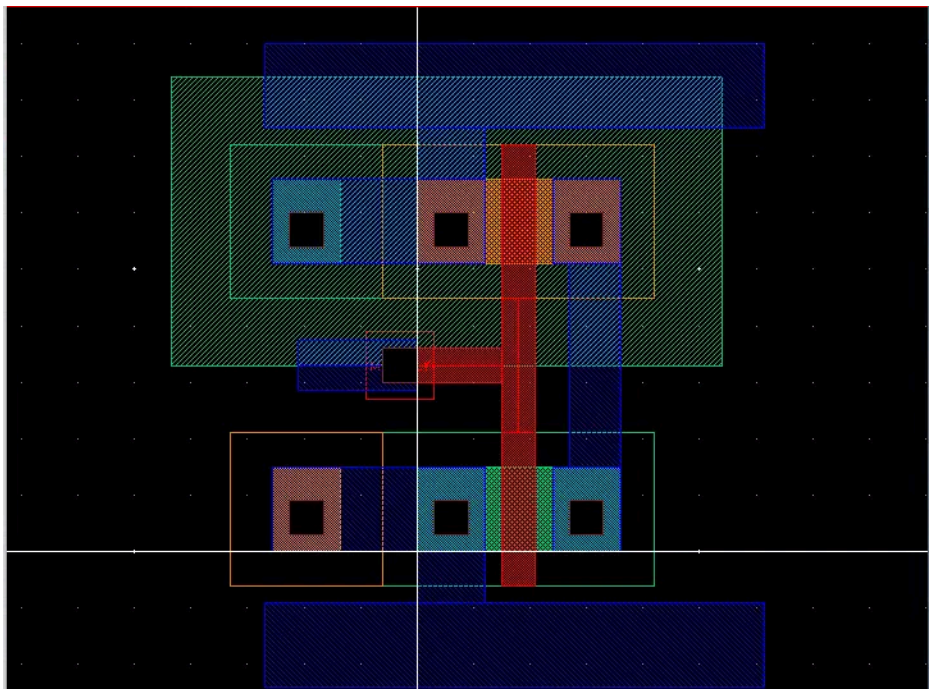

Figure 2.1 - Inverter layout after the tutorial

We first need to extract the Pmos and Nmos from the inverter and copy each of them eight times to have the required number of transistors.

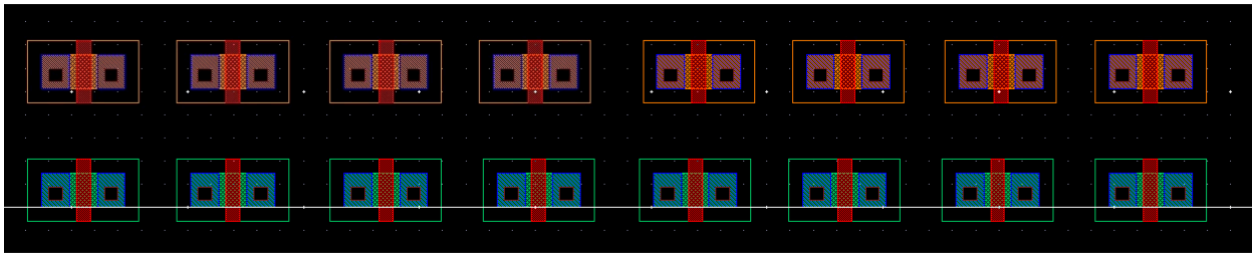

Figure 2.2 - All the transistors needed to draw the layout

Obviously, we will simplify this by removing unnecessary contact by making shared diffusion zones between Nmos transistors. Moreover, we will add the p-well contact and n-well contact and we will link transistors to achieve F. Giving us the following:

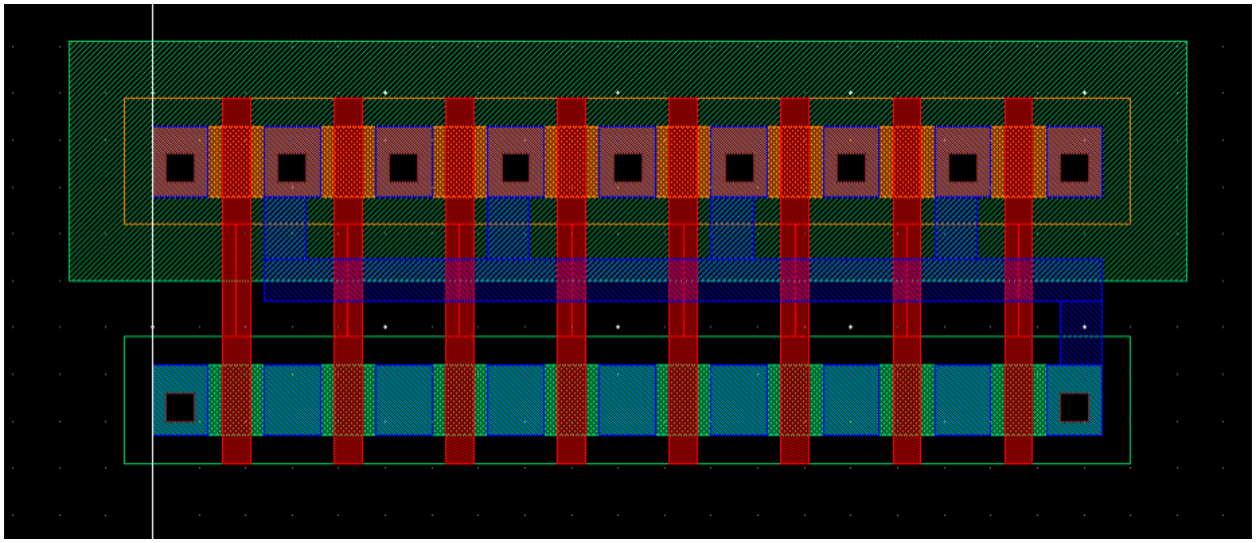

Figure 2.3 - Intermediary unsized 8-input layout without connection to GND, VDD and without vias

Since each nmos are in parallel only one of them needs to be connected to the output and another one needs to be connected to the Gnd. However, each Pmos are in parallel meaning each of them needs one contact to VDD and one contact to the output.

Now the issue is that transistors are not properly sized to match the 12u and 3u like in our schematic. So, we need to expand the layout components to match that size. Afterwards we will add the GND and VDD metal region as well as the vias in such a manner that the layout passes the DRC.

Here is the final layout:

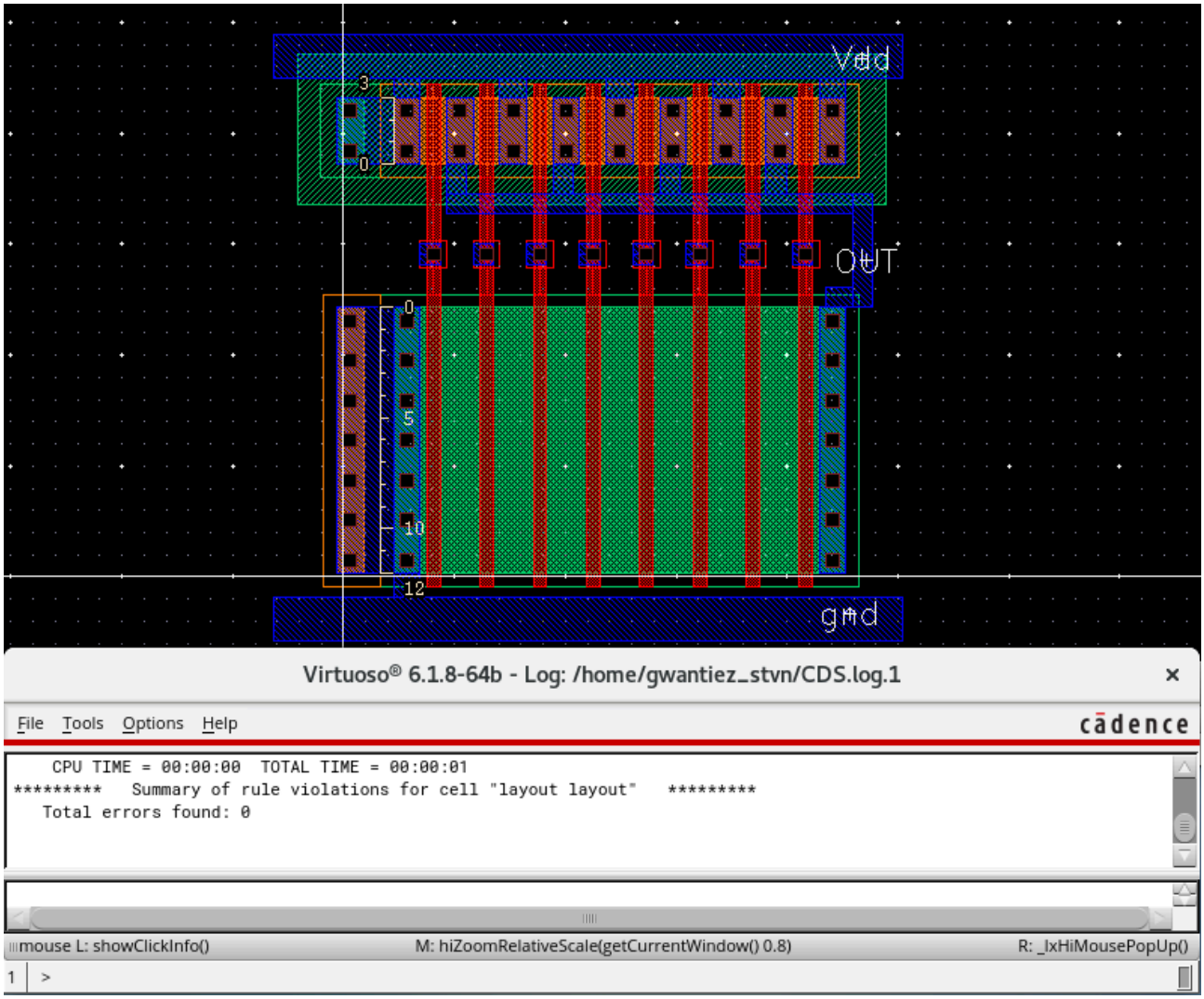

Figure 2.4 - Final 8-input layout with DRC passed successfully

Extract and LVS check

The last step is to be sure that the layout and the schematic are matching and simulate the proper behavior for our 8 input NAND gate. First let's extract our design:

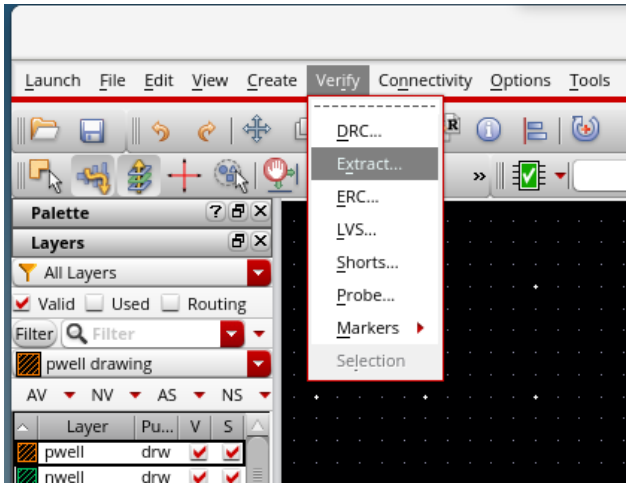

Figure 2.5 - Extract button

We get the following extracted view:

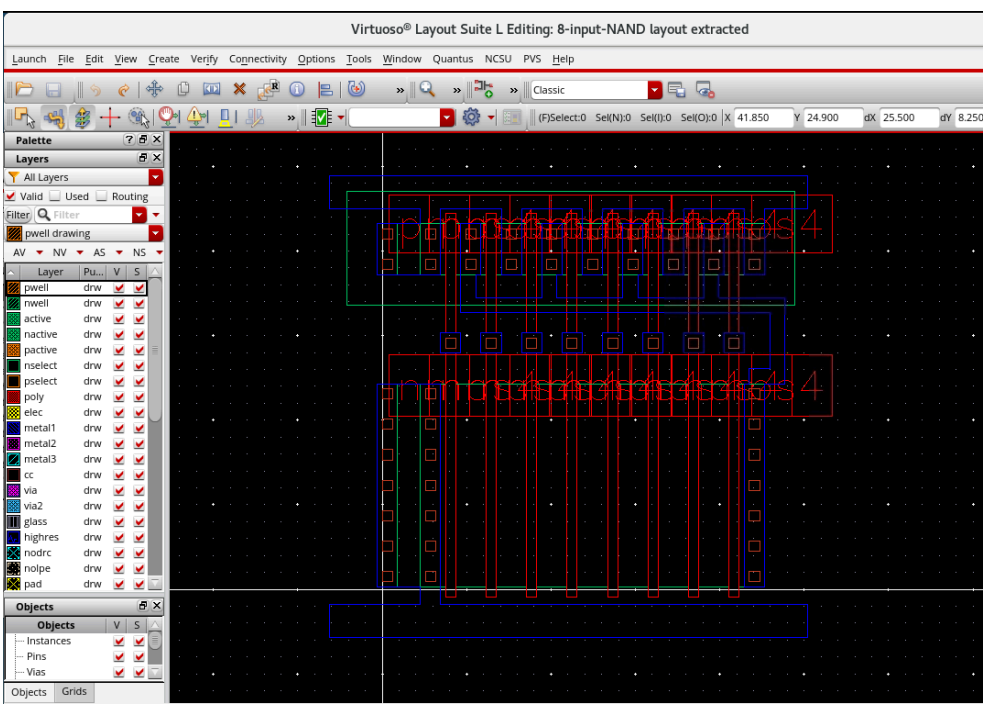

Figure 2.6 - Extracted layout

Last thing is to run the LVS with our extracted view:

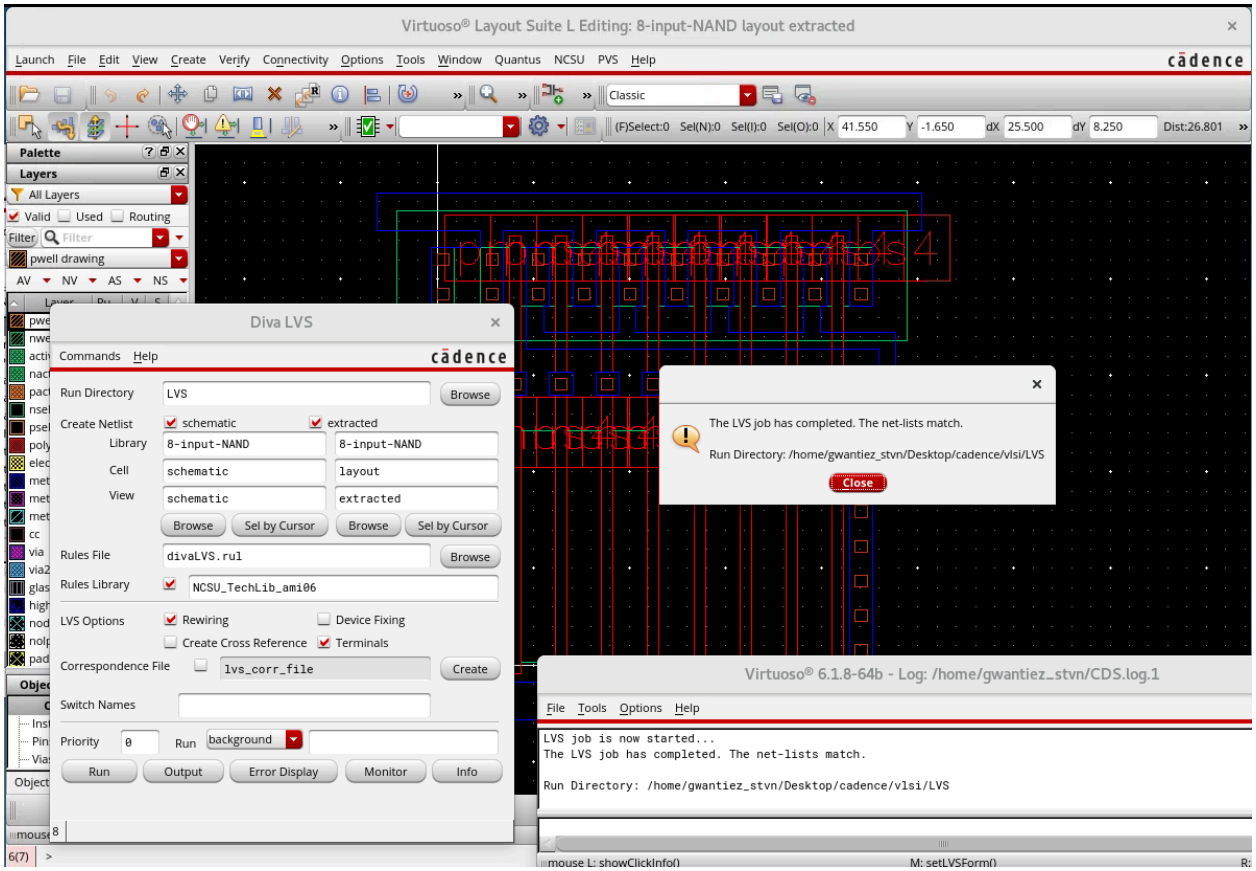

Figure 2.7 - LVS check successful, showing the schematic and the layout matching

Part 3: Delay estimation based on simulations

Now we will derive the falling delay and rising delay of our gate when driving a 10f F output load. First, we need to add to our schematic the output load.

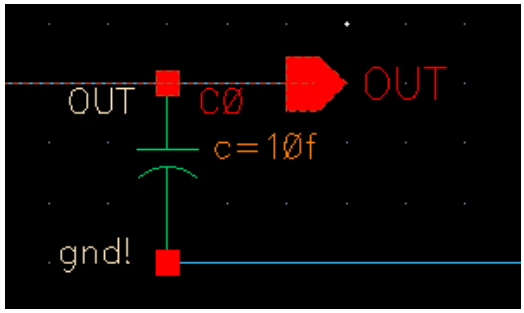

Figure 3.1 - Output capacitance of 10f F

We also need to adjust our simulation and change input feeding parameters (vpulse sources) to get proper results:

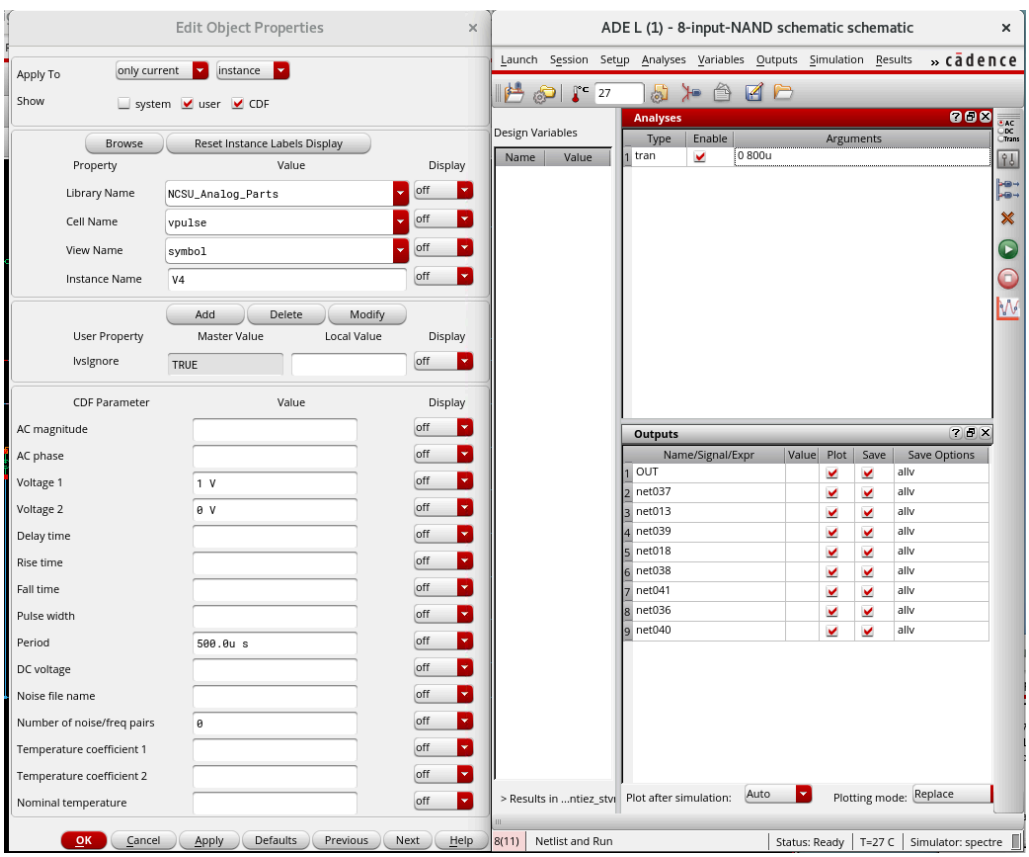

Figure 3.2 - On the left vpulse parameters for each input, on the right our simulations parameters

Falling delay when driving 10f F

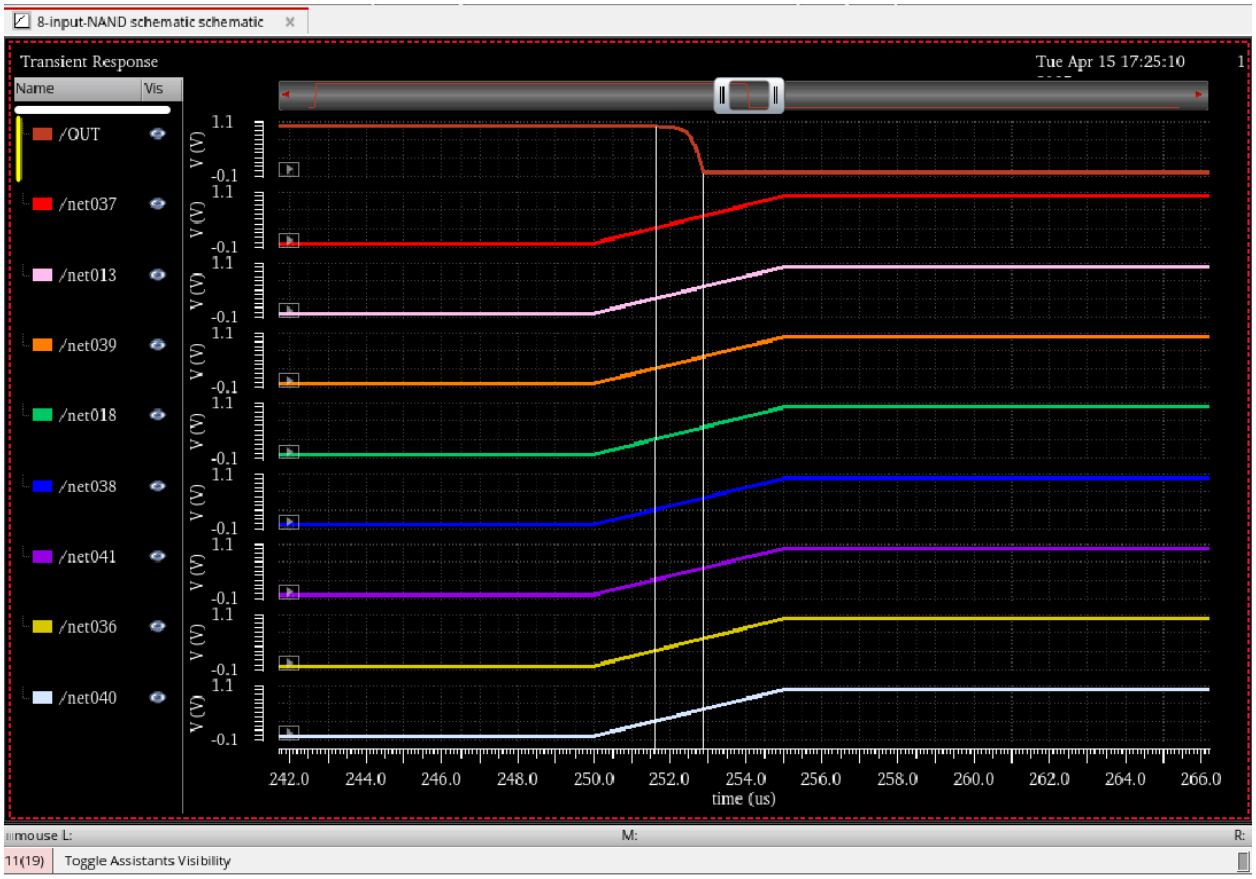

Figure 3.3 - Simulation showing the output signal (brown) and the input signals (other colors) during a falling event

Figure 3.3 are the results obtained after setting up the vpulse components (Figure 3.2) and the output load capacitance (Figure 3.1). We can observe that the output signal started to fall from 1V at 251.6ms and reached 0V at 252.9 us, giving us a falling delay of approximately 1.3us (252.9-251.6 = 1.3)

Rising and falling delay when driving 10 pF

After adjusting the output capacitance load from 10 to 1pF and relaunching the same simulation we get the following curves for a rising and a falling event:

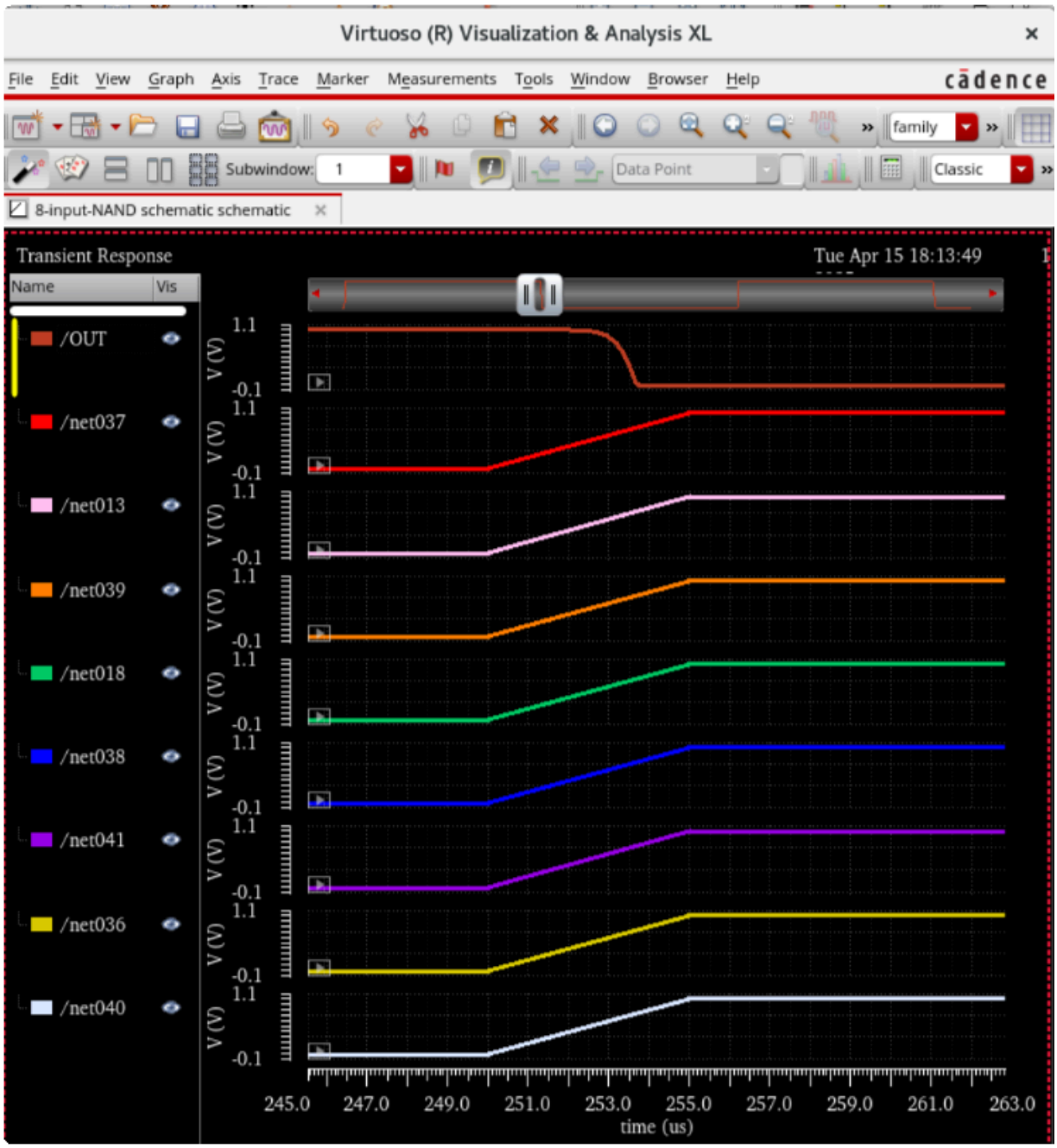

Figure 3.5 - Simulation showing the output signal (brown) and the input signals (other colors) during a falling event (load 1pF)

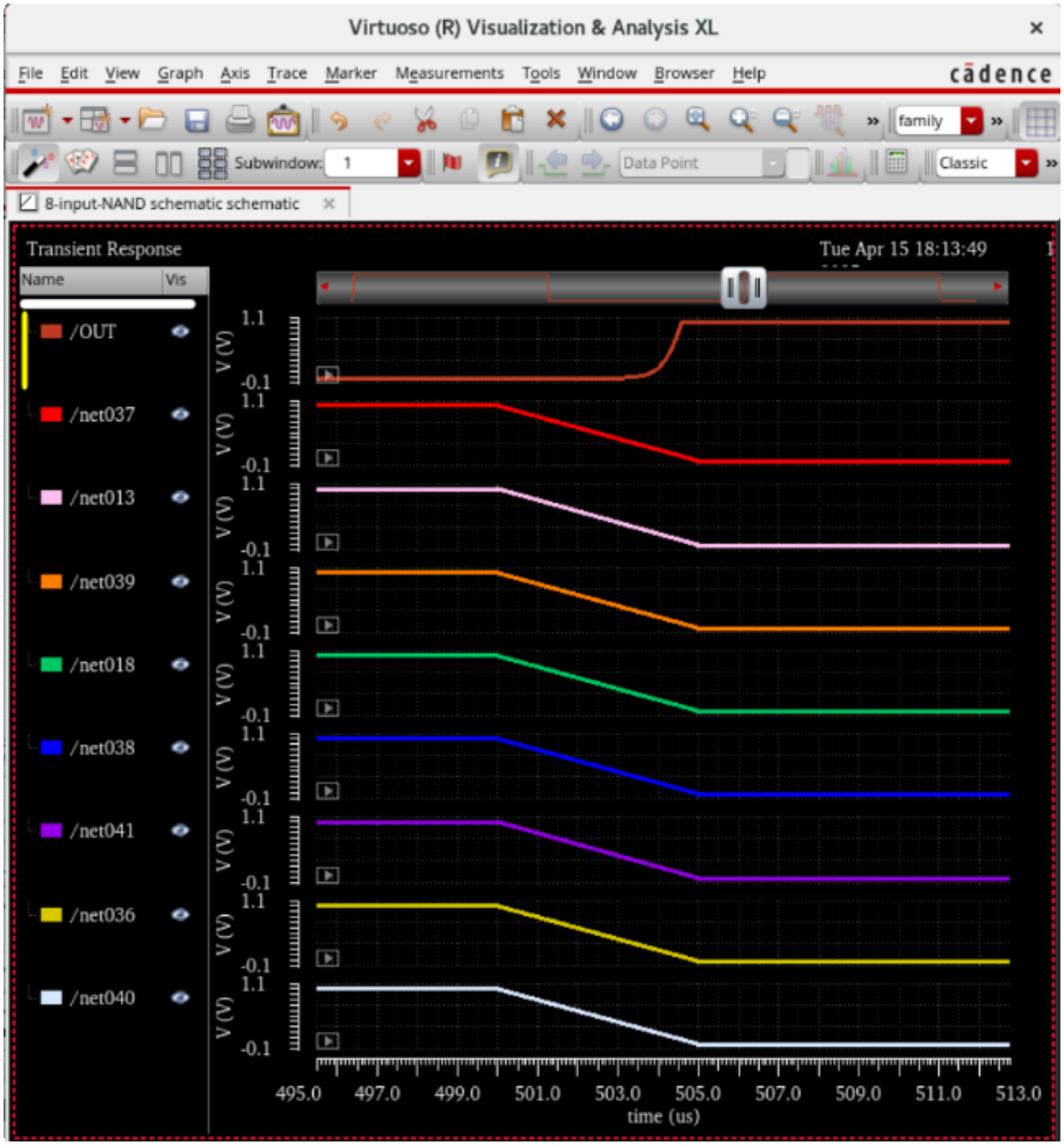

Figure 3.6 - Simulation showing the output signal (brown) and the input signals (other colors) during a rising event (load 1pF).

Again, by doing the same computations and graph analysis we obtain a falling delay of 2.3us and a rising delay of 1.6us

Let’s recap all the simulations in the table below:

	10 fF	1pF
Rising delay	0.9 us	1.6us
Falling delay	1.3 us	2.3 us

Different inputs switching comparison

These simulations are taking into account only one case (when all inputs are synchronised). Let's see if switching one input or another impacts the delay. Our intuition will be that when the farthest input to output (H) switch it will worsen the delay due to the nmos stack and the best case would be that the closest input (A) to output switch.

We will compute several simulations for rising and falling delay induced by different input, see how it impact delay time and to confirm or infirm our intuitions, to simulate a rogue input for rising delay, we will simply start all the vpulse at 1V, then we will delay 7 inputs so that only one goes to 0 while others stay at one.

Rising delay

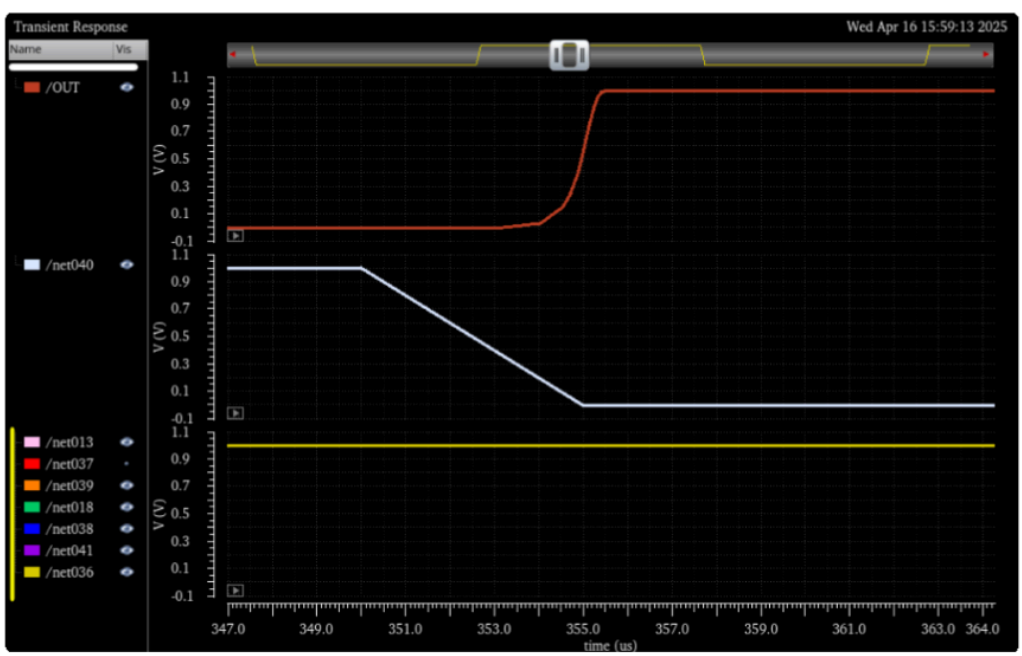

Figure 3.7 - not(A.B.C.D.E.F.G.H) with H going to 0 while others are 1

Rising delay

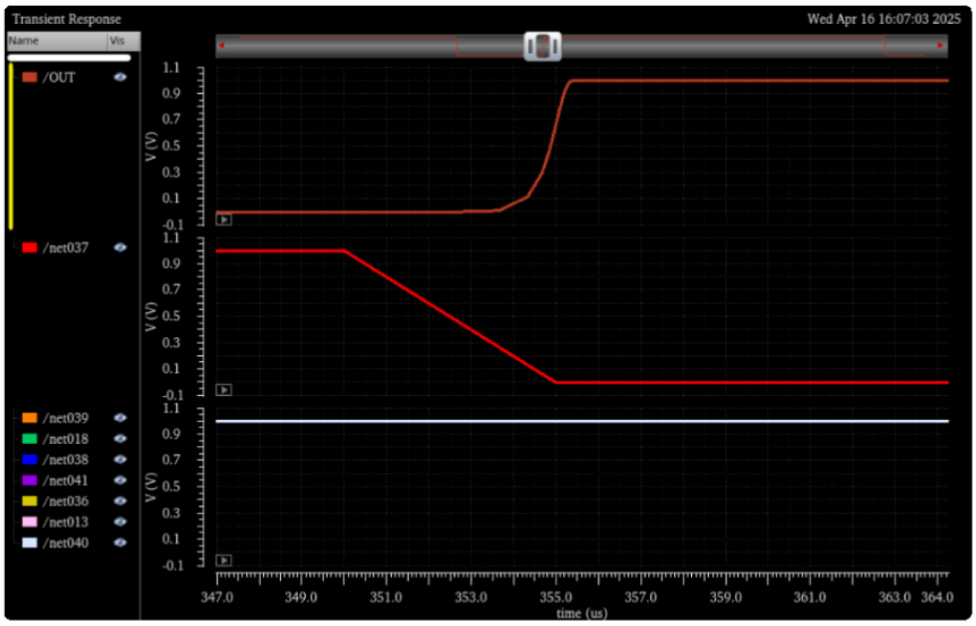

Figure 3.8 - not(A.B.C.D.E.F.G.H) with A going from 1 to 0 while others are 1

In a similar way to simulate a rogue input for falling delay, we will simply start 7 vpulse at 1V and one to 0V, then we will delay them so that only the targeted one rises while others are already at one.

Falling delay

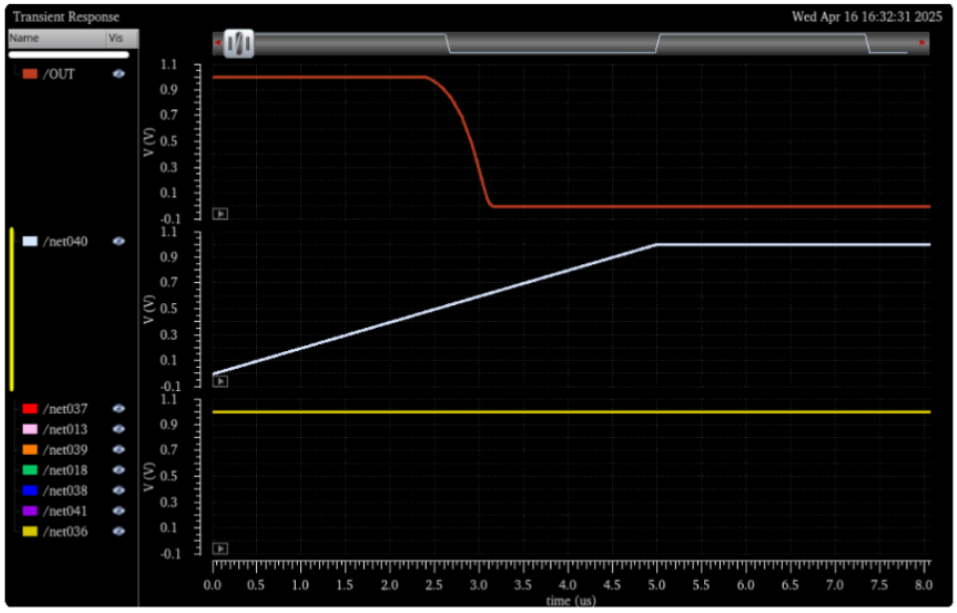

Figure 3.9 - not(A.B.C.D.E.F.G.H) with H going from 0 to 1 while others are 1

Falling delay

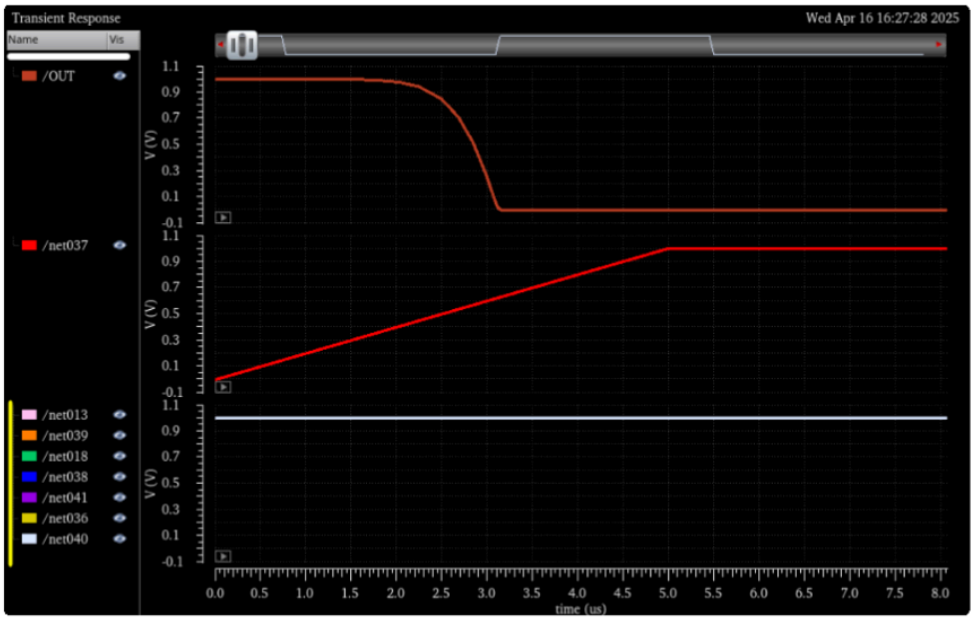

Figure 3.10 - not(A.B.C.D.E.F.G.H) with A going from 0 to 1 while others are 1

Here is the recap of the delays time found using the same calculations as earlier with the new simulations :

	(closest input to output switching) A	**(farthest input to output switches)** H
Rising delay	2.5 us	2.4 us
Falling delay	1.55 us	0.75 us

As we can see for rising delay since every pmos are in parallel, the delay is quite the same, however for falling delay, the simulation results contradict our intuition.

The results for falling delay could be due to the fact that the intermediary transistors capacitances except the one that receives the switching input are already charged (pre-set to 1V). Since the farthest is linked to the GND, the only diffusion capacitance it has to charge is on the gate and on the drain, while in the other scenario (A) the transistor have all three (gate, drain and source) diffusion capacitance to charge inducing higher possible delay.

Nmos effective resistance based on simulation result

We know that :

$$t_{pdf} = ln(2) \times R_n \times C$$

Hence, $R_n = \frac{t_{pdf}}{ln(2) \times C}$

On Figure 3.5, after graphical analysis, tpdf = 2.3 us

Finally, we get : $R_n = \frac{t_{pdf}}{ln(2) \times C} \simeq \frac{2.3 \times 10^{-6}}{0.69 \times 1 \times 10^{-12}} \simeq 2.3/0.69 \times 10^6\ \Omega \simeq 3.3\ k\Omega$

Temperature and VDD changes impact on design simulation

Here is the table of circuit speed results in differents environment with differents VDDs

	2.5V	3V	3.5V
-25C	0.66 us	0.58 us	0.51 us
25C	0.64 us	0.62 us	0.61 us
100C	0.94 us	0.86 us	0.81 us

As we can see low temperatures tend to speed up the circuit as well as high voltage, the worst average delay is achieved on low voltage and higher temperature.

Part 4: Problems encountered during the project

I encountered one major problem while implementing the project, at first in Part 1, I linked the body pins of all nmos to the source thinking that eventually it would link to the GND, of course, it was not the case and got some net list matching problem.

I also encountered DRC problems specifically with my vias and poly part of the layout, I didn't know it was possible to place via directly onto the poly section of a transistor without creating an extra little metal part just like in the tutorial. actually it worked fine and I eventually passed the DRC check using a pdf summing all the VLSI design rules available here : https://ece451web.groups.et.byu.net/cadence-help/DRC_rules.pdf

I struggled with simulation at first, but when I finally understood the graph part everything came into place eventually.

It was actually a really cool project.